

```
</> C++  
  
#include <list>  
using namespace std;  
  
list<int> li;
```

1. Insert

Front

```
</> C++  
  
li.push_front(10);
```

Back

```
</> C++  
  
li.push_back(20);
```

Iterator se beech me

⟨/⟩ C++

```
auto it = li.begin();  
advance(it, 1);      // 1 step aage  
  
li.insert(it, 15);
```

2. Update

Iterator se value change karte hain.

⟨/⟩ C++

```
auto it = li.begin();  
advance(it, 2);  
  
*it = 100;
```

3. Delete

Front

```
</> C++  
  
li.pop_front();
```

Back

```
</> C++  
  
li.pop_back();
```

Iterator se

```
</> C++  
  
auto it = li.begin();  
advance(it, 1);  
  
li.erase(it);
```

Value se

⟨⟩ C++

```
li.remove(100);
```

Ye 100 value ke **saare occurrences** delete kar dega.

4. Search

⟨⟩ C++

```
#include <algorithm>
```

```
auto it = find(li.begin(), li.end(), 20);
```

```
if(it != li.end())
```

```
    cout << "Found";
```

```
else
```

```
    cout << "Not Found";
```



5. Traverse

Range-based for

```
</> C++  
  
for(int x : li)  
    cout << x << " ";
```

Iterator

```
</> C++  
  
for(auto it = li.begin(); it != li.end(); it++)  
    cout << *it << " ";
```

Function	Work	Time
<code>push_back(x)</code>	Last me insert	$O(1)$
<code>push_front(x)</code>	Front me insert	$O(1)$
<code>pop_back()</code>	Last delete	$O(1)$
<code>pop_front()</code>	Front delete	$O(1)$
<code>insert(it,x)</code>	Iterator ke pehle insert	$O(1)$
<code>erase(it)</code>	Iterator wala node delete	$O(1)$
<code>remove(x)</code>	Value ke saare occurrences delete	$O(n)$
<code>find()</code> (<i>algorithm</i>)	Search	$O(n)$
<code>front()</code>	First element	$O(1)$
<code>back()</code>	Last element	$O(1)$



<code>back()</code>	Last element	<code>O(1)</code>
<code>size()</code>	Number of elements	<code>O(1)</code>
<code>empty()</code>	Empty hai ya nahi	<code>O(1)</code>
<code>clear()</code>	Sab delete	<code>O(n)</code>
<code>reverse()</code>	Reverse list	<code>O(n)</code>
<code>sort()</code>	Sort list	<code>O(n log n)</code>
<code>unique()</code>	Consecutive duplicates remove	<code>O(n)</code>
<code>merge(l2)</code>	Do sorted lists merge	<code>O(n+m)</code>
<code>splice()</code>	Ek list ke nodes dusri list me move	<code>O(1)</code> (iterator version)

```
#include <list>
#include <algorithm>
using namespace std;

int main() {

    list<int> li = {10,20,30};

    li.push_front(5);      // 5 10 20 30
    li.push_back(40);      // 5 10 20 30 40

    auto it = li.begin();
    advance(it,2);

    li.insert(it,15);      // 5 10 15 20 30 40

    li.erase(it);          // 5 10 15 30 40

    li.remove(30);          // 5 10 15 40

    li.reverse();           // 40 15 10 5

    li.sort();              // 5 10 15 40
```



vector STL - Most Important Functions

`</> C++`

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<int> v;
```

Operation	Function	Time
Insert End	<code>push_back(x)</code>	<code>O(1)</code> (amortize)
Remove End	<code>pop_back()</code>	<code>O(1)</code>
Insert at Position	<code>insert(it, x)</code>	<code>O(n)</code>
Delete at Position	<code>erase(it)</code>	<code>O(n)</code>
Delete Range	<code>erase(it1, it)</code> 	<code>O(n)</code>

Delete Range	<code>erase(it1, it2)</code>	<code>O(n)</code>
Update	<code>v[i] = x</code>	<code>O(1)</code>
Search	<code>find(v.begin(), v.end(), x)</code>	<code>O(n)</code>
Access	<code>v[i] / at(i)</code>	<code>O(1)</code>
First Element	<code>front()</code>	<code>O(1)</code>
Last Element	<code>back()</code>	<code>O(1)</code>
Size	<code>size()</code>	<code>O(1)</code>
Empty	<code>empty()</code>	<code>O(1)</code>
Clear	<code>clear()</code>	<code>O(n)</code>
Sort	<code>sort(v.begin(), v.end())</code>	<code>O(n log n)</code>
Reverse	<code>reverse(v.begin(), v.end())</code>	<code>O(n)</code>
Max Element	<code>max_element()</code>	<code>O(n)</code>
Min Element	<code>min_element()</code>	<code>O(n)</code>
Count	<code>count(v.begin(), v.end(), x)</code>	<code>O(n)</code>

Examples

Insert at Index

⌞ C++

```
v.insert(v.begin() + 1, 15);
```

Output:

```
10 15 20 30
```

Delete

⌞ C++

```
v.pop_back();
```

⌞ C++

```
v.erase(v.begin() + 1);
```

Update

```
</> C++
```

```
v[2] = 100;
```

Search

```
</> C++
```

```
auto it = find(v.begin(), v.end(), 20);
```

```
if(it != v.end())
```

```
    cout << "Found";
```

```
else
```

```
    cout << "Not Found";
```

Traverse

</> C++

```
for(int x : v)
    cout << x << " ";
```

or

</> C++

```
for(int i = 0; i < v.size(); i++)
    cout << v[i] << " ";
```

Sort

</> C++

```
sort(v.begin(), v.end());
```

deque STL (Double Ended Queue)

⟨⟩ C++

```
#include <iostream>
#include <deque>
#include <algorithm>
using namespace std;

deque<int> dq;
```

Operations

Operation	Function	Time
Insert Front	push_front(x)	O(1)
Insert Back	push_back(x)	O(1)
Delete Front	pop_front()	O(1)
Delete Back	pop_back()	O(1)



Insert at Position	<code>insert(it,x)</code>	<code>O(n)</code>
Delete at Position	<code>erase(it)</code>	<code>O(n)</code>
Update	<code>dq[i]=x</code>	<code>O(1)</code>
Search	<code>find(dq.begin(),dq.end(),x)</code>	<code>O(n)</code>
Access	<code>dq[i]</code> , <code>at(i)</code>	<code>O(1)</code>
First Element	<code>front()</code>	<code>O(1)</code>
Last Element	<code>back()</code>	<code>O(1)</code>
Size	<code>size()</code>	<code>O(1)</code>
Empty	<code>empty()</code>	<code>O(1)</code>
Clear	<code>clear()</code>	<code>O(n)</code>

Clear	<code>clear()</code>	<code>O(n)</code>	✦ Upgr
Sort	<code>sort(dq.begin(), dq.end())</code>	<code>O(n log n)</code>	
Reverse	<code>reverse(dq.begin(), dq.end())</code>	<code>O(n)</code>	

1. Insert

Back

⌞ C++

```
dq.push_back(10);
dq.push_back(20);
```



Front

`</> C++`

```
dq.push_front(5);
```

Output

5 10 20

2. Insert at Position

`</> C++`

```
dq.insert(dq.begin()+1,15);
```

Output

5 15 10 20



3. Delete

Front

```
</> C++  
  
dq.pop_front();
```

Back

```
</> C++  
  
dq.pop_back();
```

Position

```
</> C++  
  
dq.erase(dq.begin()+1);
```

4. Update

```
</> C++  
  
dq[2]=100;
```

or

```
</> C++  
  
dq.at(2)=100;
```

5. Search

```
</> C++  
  
auto it = find(dq.begin(),dq.end(),20);  
  
if(it!=dq.end())  
    cout<<"Found";  
else  
    cout<<"Not Found";
```

6. Traverse

Range-based for

```
</> C++  
  
for(int x:dq)  
    cout<<x<<" ";
```

Normal for

```
</> C++
```

```
for(int i=0;i<dq.size();i++)  
    cout<<dq[i]<<" ";
```

Iterator

```
</> C++
```

```
for(auto it=dq.begin();it!=dq.end();it++)  
    cout<<*it<<" ";
```